

Unsupervised Learning

Unit -2

Unsupervised Learning

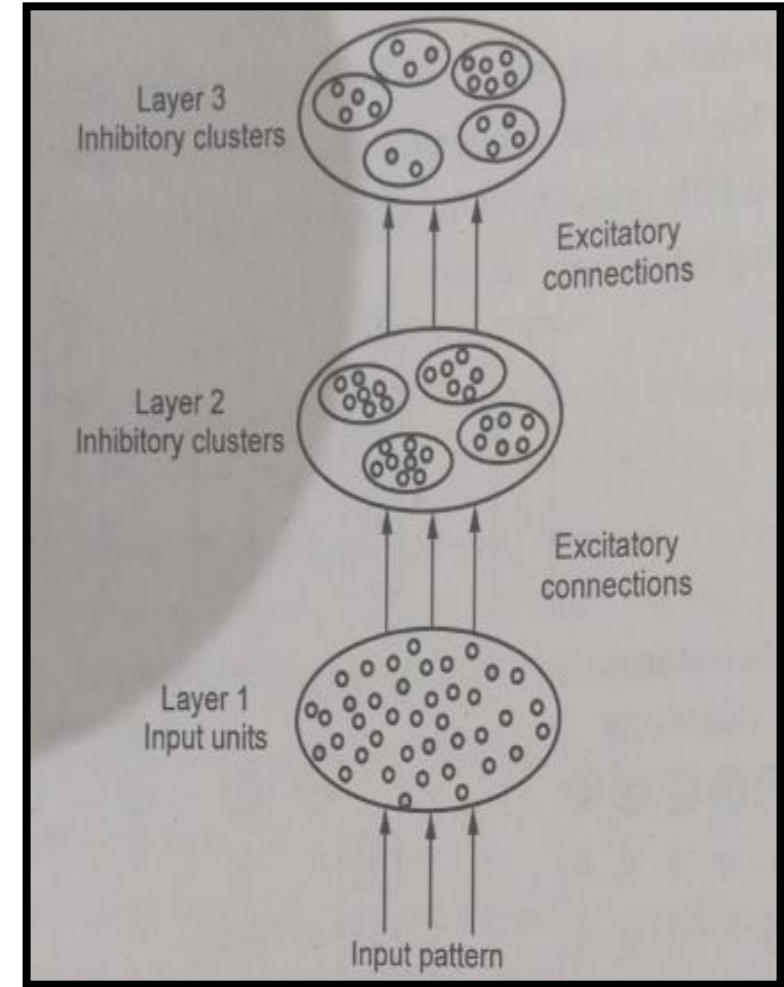
- Clustering
- Simple competitive learning algorithm
- LVQ algorithm
- SOM
- Adaptive Resonance Theory

Clustering

- It refers to the idea of grouping similar data points based on learned features or patterns within a neural network.
- This concept is useful in tasks like dimensionality reduction, unsupervised learning, or understanding the internal representation of data within the network.
- This concept plays a crucial role in tasks like data grouping, anomaly detection, segmentation, and visualizing patterns in high-dimensional data.

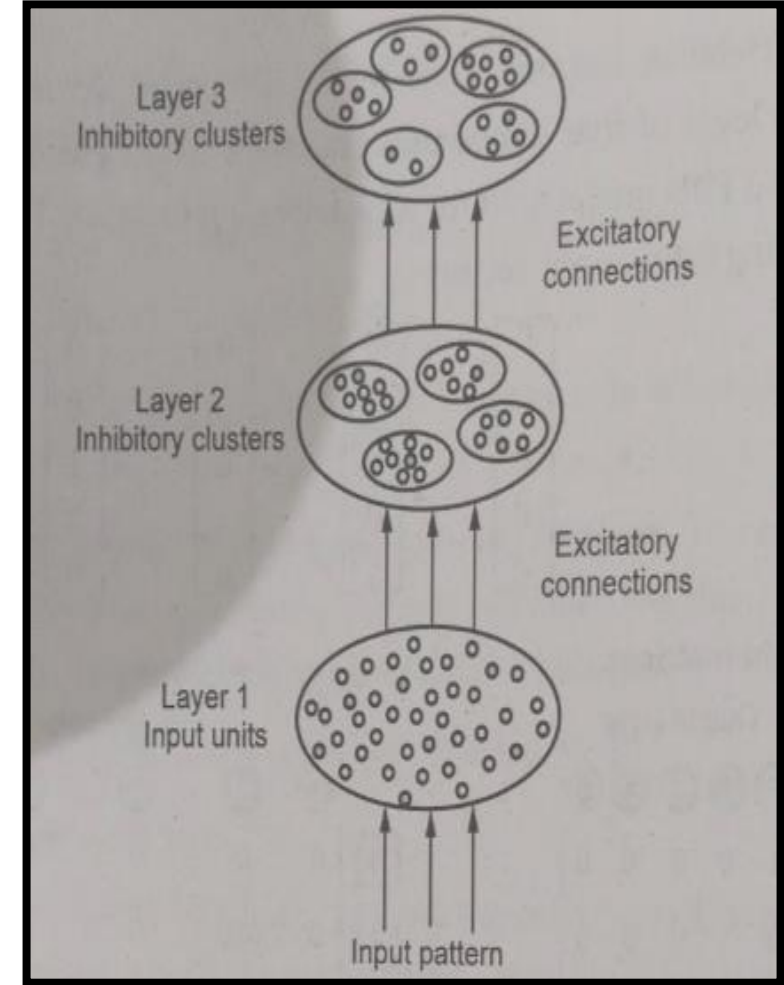
Simple Competitive Learning Algorithm

- It consists of a set of **hierarchically layered units** in which each layer connects, via **excitatory connections**, with the layer immediately above it, and has inhibitory connections to units in its own layer.
- Each unit in a layer receives an input from each unit in the layer immediately below it and projects to each unit in the layer immediately above it.
- Moreover, **within a layer**, the units are broken into a set of inhibitory clusters in which all elements within a cluster inhibit all other elements in the cluster.



Simple Competitive Learning Algorithm

- Thus the elements within a cluster at one level compete with one another to respond to the pattern appearing on the layer below.
- The more strongly any particular unit responds to an incoming stimulus, the more it shuts down the other members of its cluster.
- Units are represented in the diagram as dots. Units may be active or inactive . Active units are represented by filled dots, inactive ones by open dots.



Properties

- The units in a given layer are broken into several sets of clusters. Each unit within a cluster inhibits every other unit. Within each cluster, the unit receiving the largest input achieves its maximum value while all other units in the cluster are pushed to their minimum value 1.
- A unit learns if and only if it wins the competition with other units in its cluster.
- Each unit has a fixed amount of weight (all weights are positive) that is distributed among its input lines.

Characteristics

- Each cluster classifies the stimulus set into M groups, one for each unit in the cluster. Each of the units captures roughly an equal number of stimulus patterns.
- If there is structure in the stimulus patterns, the units will break up the patterns along structurally relevant lines.
- The particular grouping done by a particular cluster depends on the starting value of the weights and the sequence of stimulus patterns actually presented.
- **To a first approximation, the system develops clusters that minimize within-cluster distance, maximize between-cluster distance, and balance the number of patterns captured by each cluster**

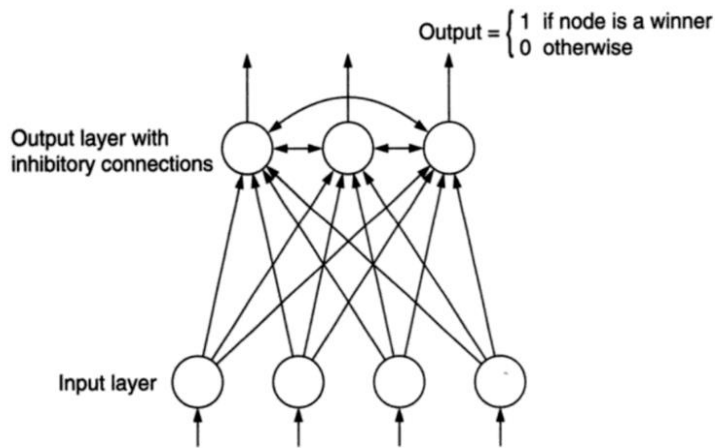


Figure
Simple competitive learning network.

Algorithm Simple Competitive Learning;

Initialize all weights to random values;

repeat

- (Optional:) Adjust the learning rate $\eta(t)$;
- Select an input pattern i_k from the training set;
- Find node j^* whose weight vector w_{j^*} is closest to i_k ;
- Update each weight $w_{j^*,1}, \dots, w_{j^*,n}$ using the rule:

$$\Delta w_{j^*,\ell} = \eta(t)(i_{k,\ell} - w_{j^*,\ell}), \text{ for } \ell \in \{1, \dots, n\}$$

until network converges or computational bounds are exceeded.

Cluster the following points using Simple Competitive Learning $X_1 = (1 \ 1 \ 1)^T$,
 $X_2 = (0.5 \ 0.7 \ 0.8)^T$, $X_3 = (0.2 \ 0.3 \ 0.4)^T$ and $X_4 = (0.1 \ 0.1 \ 0.2)^T$

Assume learning constant as 0.5 and initial weights as $W = 0.2 \ 0.4 \ 0.3$

0.8 0.9 0.1

0.5 0.6 0.3

- Given 3-d vectors:

$$X_1 = [1 \ 1 \ 1]^T$$

$$X_2 = [0.5 \ 0.7 \ 0.8]^T$$

$$X_3 = [0.2 \ 0.3 \ 0.4]^T$$

$$X_4 = [0.1 \ 0.1 \ 0.2]^T$$

Learning constant $\alpha = 0.5$

Initial Weight Matrix $W =$

0.2	0.4	0.3
0.8	0.9	0.1
0.5	0.6	0.3

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- Step 1: Compute the Euclidean distance.

Between each input vector and each weight vector, then update the weights of the winning neuron (i.e., the one with the smallest distance).

$$\mathbf{X1} = [1 \ 1 \ 1]^T \quad \mathbf{W1} = [0.2 \quad 0.4 \quad 0.3]$$

- Distance from X1 to W1

- $D(\mathbf{X1}, \mathbf{W1}) = \sqrt{(0.2 - 1)^2 + (0.4 - 1)^2 + (0.3 - 1)^2} = \sqrt{1.49} = 1.22$

- Distance from X1 to W2

- $D(\mathbf{X1}, \mathbf{W2}) = \sqrt{0.86} = 0.927$

- Distance from X1 to W3

- $D(\mathbf{X1}, \mathbf{W3}) = \sqrt{0.9} = 0.949$

- The smallest distance is 0.927 for Neuron 2.

Update the Weights wrt X1

$$W_2(t+1) = W_2(t) + \alpha (X1 - W_2(t))$$

$$\begin{aligned} W_2(t+1) &= [0.8, 0.9, 0.1] + (0.5 * ([1, 1, 1] - [0.8, 0.9, 0.1])) \\ &= [0.8, 0.9, 0.1] + (0.5 * [0.2, 0.1, 0.9]) \\ &= [0.9, 0.95, 0.55] \end{aligned}$$

Updated Weight Matrix W =

0.2	0.4	0.3
0.9	0.95	0.55
0.5	0.6	0.3

- Compute the Euclidean distance between X2 and Updated Weight Matrix.
- Distance from X2 to W2
- $D(X2, W1) = \sqrt{0.43} = 0.656$
- Distance from X2 to W2
- $D(X2, W2) = \sqrt{0.285} = 0.534$
- Distance from X2 to W3
- $D(X2, W3) = \sqrt{0.26} = 0.51$
- The smallest distance is 0.51 for Neuron 3.

Update the Weights wrt X2

$$W_3(t+1) = W_3(t) + \alpha (X2 - W_3(t))$$

$$W_3(t+1) = [0.5, 0.65, 0.55]$$

Updated Weight Matrix W =	0.2	0.4	0.3
	0.9	0.95	0.55
	0.5	0.65	0.55

- Compute the Euclidean distance between X3 and Updated Weight Matrix.
- Distance from X3 to W2
- $D(X3, W1) = \sqrt{0.02} = 0.141$
- Distance from X3 to W2
- $D(X3, W2) = \sqrt{0.935} = 0.967$
- Distance from X3 to W3
- $D(X3, W3) = \sqrt{0.235} = 0.485$
- The smallest distance is 0.141 for Neuron 1.

Update the Weights wrt X3

$$W_1(t+1) = W_1(t) + \alpha (X3 - W_1(t))$$

$$W_1(t+1) = [0.2, 0.35, 0.35]$$

Updated Weight Matrix W =

0.2	0.35	0.35
-----	------	------

0.9	0.95	0.55
-----	------	------

0.5	0.65	0.55
-----	------	------

- Compute the Euclidean distance between X4 and Updated Weight Matrix.
- Distance from X4 to W2
- $D(X4, W1) = \sqrt{0.095} = 0.308$
- Distance from X4 to W2
- $D(X4, W2) = \sqrt{1.485} = 1.218$
- Distance from X4 to W3
- $D(X4, W3) = \sqrt{0.585} = 0.765$
- The smallest distance is 0.308 for Neuron 1.

Update the Weights wrt X4

$$W_1(t+1) = W_1(t) + \alpha (X4 - W_1(t))$$

$$W_1(t+1) = [0.15, 0.225, 0.275]$$

Updated Weight Matrix W =

0.15	0.225	0.275
------	-------	-------

0.9	0.95	0.55
-----	------	------

0.5	0.65	0.55
-----	------	------

Final Output for SCL

Weight Matrix W =

0.15	0.225	0.275
0.9	0.95	0.55
0.5	0.65	0.55

- Neuron 1 → X3 , X4
- Neuron 2 → X1
- Neuron 3 → X2

Advantages of SCL

Biologically Inspired:

Simple Competitive Learning (SCL) is inspired by biological systems, particularly neural networks, where neurons compete to represent the input. It simulates how biological neurons compete for activation.

Simplicity:

The algorithm is easy to implement and computationally efficient, especially for small datasets or low-dimensional data.

It involves simple updates based on a winning neuron and can be executed quickly compared to more complex algorithms.

Effective for Topology-Based Learning:

SCL is often used in **self-organizing maps (SOM)**, where preserving topological relationships between data points is important. It can help map high-dimensional data to a lower-dimensional grid while maintaining the relationships between points.

Advantages of SCL

Good for Clustering:

The algorithm can automatically group similar inputs together based on weight adjustment. It is useful for unsupervised learning tasks like clustering, especially when there's no prior knowledge about the number of clusters.

Incremental Learning:

SCL adapts the weights based on each input iteratively. It is suitable for **online learning**, where the data arrives sequentially and the model can be updated incrementally.

Does Not Require a Predefined Number of Classes:

Unlike K-means, SCL does not require predefining the number of clusters/classes. The neurons' weights self-organize to represent different regions of the input space.

Disadvantages of SCL

Sensitive to Initial Conditions:

The final outcome of the competitive learning process depends significantly on the initial values of the weights. Poor initialization can lead to suboptimal solutions where certain neurons do not "win" and fail to adjust their weights effectively.

Winner-Takes-All Strategy:

The algorithm updates only the **winning neuron** (the closest neuron to the input). This can cause problems when multiple neurons need to represent similar input patterns. The result may lead to underrepresentation of some patterns and overrepresentation of others.

No Direct Control Over Cluster Size:

Since SCL updates only the winning neuron, there is no explicit control over how much space each neuron should represent. Some neurons may end up representing a larger portion of the input space, while others may represent smaller or more specific regions, potentially leading to imbalanced clustering.

Fixed Learning Rate:

The learning rate can greatly affect convergence. A fixed or poorly chosen learning rate may cause the algorithm to either converge too slowly (if too small) or oscillate and fail to stabilize (if too large). In practice, the learning rate often needs to decrease over time, which is not inherent in the algorithm.

Disadvantages of SCL

Does Not Handle Non-Linearly Separable Data Well:

Like many clustering algorithms, simple competitive learning may struggle with **non-linearly separable** data unless significant modifications are made.

Lack of Global Optimization:

SCL does not use a global criterion for convergence like K-means or other optimization-based algorithms. There is no clear objective function (such as minimizing total within-cluster variance) that ensures the clustering process is moving toward an optimal solution.

May Lead to "Dead Neurons":

Some neurons might never win, leading to **dead neurons**, which never update their weights and therefore don't contribute to the learning process. This is particularly a problem if the initial weight vector is far from any input vectors.

Convergence Issues:

The algorithm may converge slowly, or even not at all, if the learning rate or initial weights are not chosen properly. Additionally, competitive learning doesn't have a formal convergence criterion, so it is hard to determine when the learning process should stop.

Sensitive to Input Data Distribution:

The algorithm's performance can degrade if the input data is not evenly distributed. If one part of the input space has a higher density of points, neurons representing this area may dominate the learning process.

LVQ Algorithm

- LVQ allows to choose the number of training instances to suspend and know exactly what these examples should look like.
- Learning vector quantization (LVQ) is a process of classifying the patterns, wherein each output unit represents a particular class.
- Here, for each class several units should be used.
- **The output unit weight vector is called the reference vector or code book vector for the class which the unit represents. This is a special case of competitive net, which uses supervised learning methodology.**

LVQ Algorithm

- **The set of training patterns with known classifications is given to the network, along with an initial distribution of the reference vectors.**
- **When the training process is complete, an LVQ net is found to classify an input vector by assigning it to the same class as that of the output unit, which has its weight vector very close to the input vector.**
- **Thus, LVQ is a classifier paradigm that adjusts the boundaries between categories to minimize existing misclassification.**
- LVQ is used for optical character recognition, converting speech into phonemes and other applications as well.

LVQ Algorithm

Algorithm LVQ1;

Initialize all weights to random values in the range $[0,1]$;

repeat

Adjust the learning rate $\eta(t)$;

for each input pattern i_k in the training set, **do**

find node j^* whose weight vector w_{j^*} is closest to i_k ;

for $\ell = 1, \dots, n$, **do**

Update the weight $w_{j^*,\ell}$ as follows:

if the class label of node j^* equals
the desired class of i_k ,

then $\Delta w_{j^*,\ell} = \eta(t)(i_{k,\ell} - w_{j^*,\ell})$

else $\Delta w_{j^*,\ell} = -\eta(t)(i_{k,\ell} - w_{j^*,\ell})$

end-for

end-for

until network converges or computational bounds are exceeded.

Algorithm Steps - Concept

- For each training example, compute the distance between the example and all weight vectors.
- Assign the sample to the weight vector (node) that has the smallest distance.
- Update the winning weight vector:
 - If the classification is correct, adjust the winning weight vector to move closer to the training point.
 - If the classification is incorrect, adjust the weight vector to move away from the training point.

Algorithm Steps - Concept

- The update formula for the weight vector w at time t is:

$$w(t+1) = w(t) + \eta(t) \times (x - w(t))$$

if classification is correct

$$w(t+1) = w(t) - \eta(t) \times (x - w(t))$$

if classification is incorrect

LVQ Numerical Example

- The data are cast into classification problem by arbitrarily associating the first and last samples with Class 1 and the rest with Class 2. Run LVQ algorithm on the given dataset till 12 iterations.
- Let $\eta(t) = 0.5$ till $t = 6$
 $\eta(t) = 0.25$ till $t = 12$
 $\eta(t) = 0.1$ thereafter
- Consider $X1 = [1.1 , 1.7 , 1.8]$ $X2 = [0 , 0 , 0]$
 $X3 = [0 , 0.5 , 1.5]$ $X4 = [1 , 0 , 0]$
 $X5 = [0.5 , 0.5 , 0.5]$ $X6 = [1 , 1 , 1]$

Initial Weight Matrix W =

w1	0.2	0.7	0.3
w2	0.1	0.1	0.9
w3	1	1	1

- **Training Set:**

- ❖ $X_1 \rightarrow \text{Class 1}$
- ❖ $X_2 \rightarrow \text{Class 2}$
- ❖ $X_3 \rightarrow \text{Class 2}$
- ❖ $X_4 \rightarrow \text{Class 2}$
- ❖ $X_5 \rightarrow \text{Class 2}$
- ❖ $X_6 \rightarrow \text{Class 1}$

- **Initial Weights:**

- ❖ $W_1 = [0.2, 0.7, 0.3] \rightarrow \text{associated with Class 1}$
- ❖ $W_2 = [0.1, 0.1, 0.9] \rightarrow \text{associated with Class 2}$
- ❖ $W_3 = [1, 1, 1] \rightarrow \text{associated with Class 2}$

- **Learning Rate:**

- ❖ $\eta(t) = 0.5$ till $t = 6$
- ❖ $\eta(t) = 0.25$ till $t = 12$
- ❖ $\eta(t) = 0.1$ > 12

Iteration 1: Compute the Euclidean distance between X1 and (W1 W2 W3) respectively.

- $D(X1, W1) = 1.9$
- $D(X1, W2) = 2.0$
- $D(X1, W3) = 0.86$
- The **closest** weight vector is **w3** belonging to (Class 2), but since **X1 is in Class 1**, the **classification is incorrect**. Therefore, we move w3 **away** from X1.

$$w3(t+1) = w3(t) - 0.5 \times ((1.1, 1.7, 1.8) - (1, 1, 1)) \\ = (1, 1, 1) - 0.5 \times (0.1, 0.7, 0.8)$$

$$w3(t+1) = (0.95 , 0.65 , 0.6)$$

❖ $W1 = [0.2 , 0.7, 0.3]$
❖ $W2 = [0.1 , 0.1 , 0.9]$
❖ $W3 = [0.95 , 0.65 , 0.60]$

Iteration 2: Compute the Euclidean distance between X_2 and $(W_1 \ W_2 \ W_3)$ respectively.

- $D(X_2, W_1) = 0.78$
- $D(X_2, W_2) = 0.91$
- $D(X_2, W_3) = 1.25$
- The **closest** weight vector is **w_1 belonging to (Class 1)**,
but since **X_2 is in Class 2**, the **classification is incorrect**.
Therefore, we move w_1 **away** from X_2 .

$$w_1(t+1) = (0.3 , 1.05 , 0.45)$$

$$\diamond W_1 = [0.3 , 1.05 , 0.45]$$

$$\diamond W_2 = [0.1 , 0.1 , 0.9]$$

$$\diamond W_3 = [0.95 , 0.65 , 0.60]$$

Iteration 3: Compute the Euclidean distance between X3 and (W1 W2 W3) respectively.

- $D(X3, W1) = 1.20$
- $D(X3, W2) = 1.00$
- $D(X3, W3) = 1.09$
- The **closest** weight vector is **w2** belonging to (Class 2), but since **X3 is in Class 2**, the **classification is correct**.

Therefore, we move w2 **closer** from X3.

$$\begin{aligned} w2(t+1) &= w2(t) + 0.5 \times ((0,0.5,1.5)-(0.1,0.1,0.9)) \\ &= (0.1,0.1,0.9)+0.5 \times (-0.1,0.4,0.6) \end{aligned}$$

$$w2(t+1) = (0.05 , 0.3 , 1.2)$$

$$\diamond W1=[0.3 , 1.05 , 0.45]$$

$$\diamond W2=[0.05 , 0.3 , 1.2]$$

$$\diamond W3=[0.95 , 0.65 , 0.60]$$

Iteration 4: Compute the Euclidean distance between X_4 and $(W_1 \ W_2 \ W_3)$ respectively.

- $D(X_4, W_1) = 1.33$
- $D(X_4, W_2) = 1.50$
- $D(X_4, W_3) = 0.84$
- The **closest** weight vector is **w_3 belonging to (Class 2)**,
but since **X_4 is in Class 2**, the **classification is correct**.

Therefore, we move w_3 **closer** from X_4 .

$$w_3(t+1) = (0.975 , 0.325 , 0.3)$$

$$\begin{aligned} \diamond W_1 &= [0.3 , 1.05 , 0.45] \\ \diamond W_2 &= [0.05 , 0.3 , 1.2] \\ \diamond W_3 &= [0.975 , 0.325 , 0.3] \end{aligned}$$

- Iteration 5: $w_3 : [0.74 \quad , \quad 0.41 \quad , \quad 0.40]$
- Iteration 6: $w_3 : [0.61 \quad , \quad 0.12 \quad , \quad 0.10]$
- Iteration 7: $w_1 : [0.50 \quad , \quad 1.21 \quad , \quad 0.79]$
- Iteration 8: $w_3 : [0.45 \quad , \quad 0.09 \quad , \quad 0.08]$
- Iteration 9: $w_2 : [0.04 \quad , \quad 0.35 \quad , \quad 1.27]$
- Iteration 10: $w_3 : [0.59 \quad , \quad 0.07 \quad , \quad 0.06]$
- Iteration 11: $w_3 : [0.57 \quad , \quad 0.18 \quad , \quad 0.17]$
- Iteration 12: $w_1 : [0.62 \quad , \quad 1.16 \quad , \quad 0.84]$

Advantages of LVQ

- **Efficient for Small to Medium-Sized Datasets**
- LVQ can perform well with small to medium-sized datasets. It's computationally efficient when dealing with these sizes since it only updates a limited number of prototype vectors during training.
- **Adaptable to Different Distance Metrics**
- LVQ can be adapted to use different distance metrics (e.g., Euclidean distance, Manhattan distance), making it flexible for different types of datasets.
- **Competitive Learning**
- LVQ employs competitive learning, where only the "winning" prototype is updated during training, which can lead to faster convergence compared to fully connected models like multilayer perceptron (MLP).
- **Simple and Intuitive**
- LVQ is relatively simple to understand and implement. The algorithm works by adjusting the prototype vectors to better represent the data points in each class, making it easier to classify new inputs.

Disadvantages of LVQ

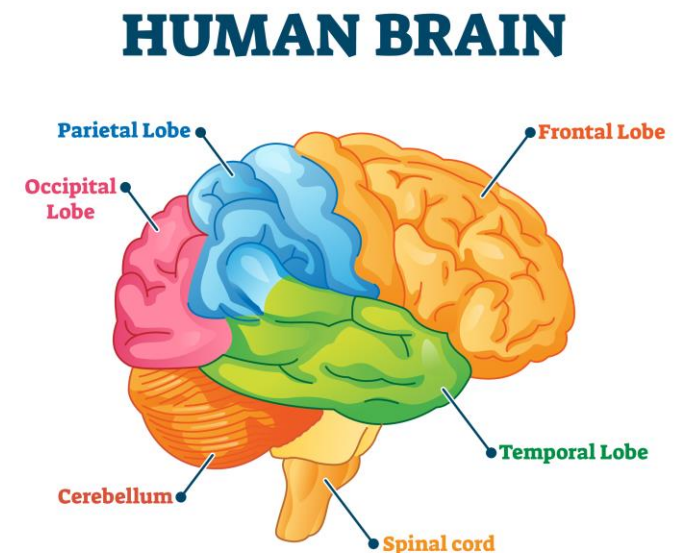
- **Sensitive to Noisy Data**
 - LVQ can be sensitive to noisy data or outliers. Since it relies on distance-based measures, it may incorrectly adjust the prototype vectors based on noisy or irrelevant data points.
- **Limited to Linearly Separable Data**
 - LVQ works best when the data is linearly separable. For complex decision boundaries, LVQ may struggle to achieve high classification accuracy without additional preprocessing or modifications
- **Sensitive to Initial Prototypes**
 - The performance of LVQ is highly sensitive to the initial positions of the prototype vectors. Poor initialization can lead to suboptimal results, and finding good initial prototypes can be challenging.
- **Prone to Local Minima**
 - Like many learning algorithms, LVQ can get stuck in local minima during optimization. As a result, the final classification may not be globally optimal.

Self Organizing Maps (SOM)

- A special class of artificial neural networks known as self-organizing maps. These networks are based on competitive learning.
- The output neurons of the network compete among themselves to be activated or fired, with the result that only one output neuron, or one neuron per group, is **ON** at any one time.
- An output neuron that wins the competition is called a **winner-takes-all** neuron, or simply a **winning neuron**.

- In a self-organizing map, the neurons are placed at the nodes of a lattice that is usually one or two dimensional.
- The neurons become selectively tuned to various input patterns (stimuli) or classes of input patterns in the course of a competitive-learning process.
- The locations of the neurons so tuned (i.e., the winning neurons) become ordered with respect to each other in such a way that a meaningful coordinate system for different input features is created over the lattice.

- A self-organizing map is therefore characterized by the formation of a topographic map of the input patterns, in which the spatial locations (i.e., coordinates) of the neurons in the lattice are indicative of intrinsic statistical features contained in the input patterns—hence, the name “self-organizing map.”
- The self-organizing map is inherently nonlinear.
- The development of self-organizing maps as a neural model is motivated by a distinct feature of the human brain:
 - The brain is organized in many places in such a way that different sensory inputs are represented by topologically ordered computational maps.

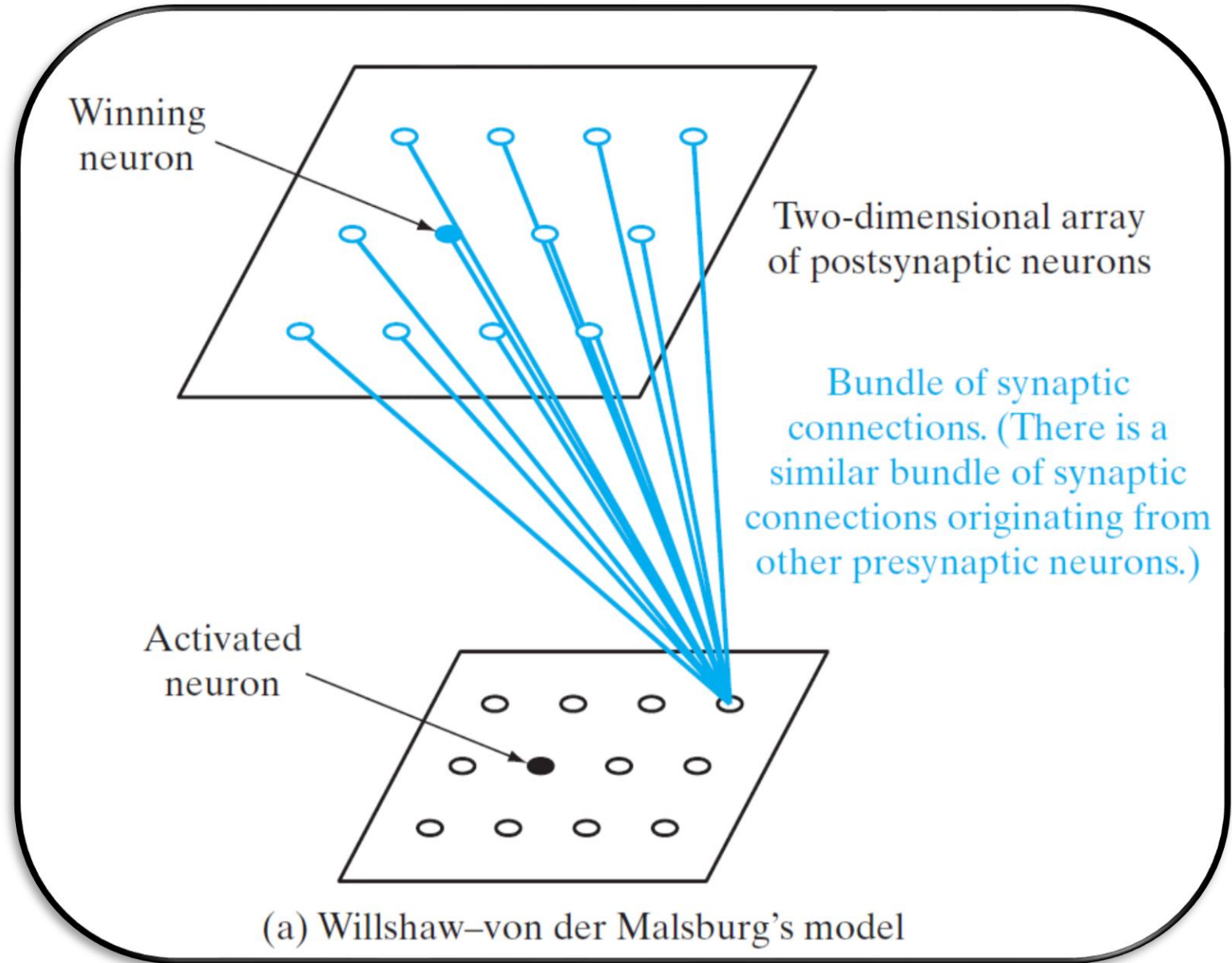


- In particular, sensory inputs such as tactile, visual and acoustic inputs are mapped onto different areas of the cerebral cortex in a topologically ordered manner.
- Thus, the **computational map** constitutes a basic building block in the information-processing infrastructure of the nervous system.
- A computational map is defined by an array of neurons representing slightly differently tuned processors or filters, which operate on the sensory information bearing signals in parallel.
- Consequently, the neurons transform input signals into a place-coded probability distribution that represents the computed values of parameters by sites of maximum relative activity within the map.

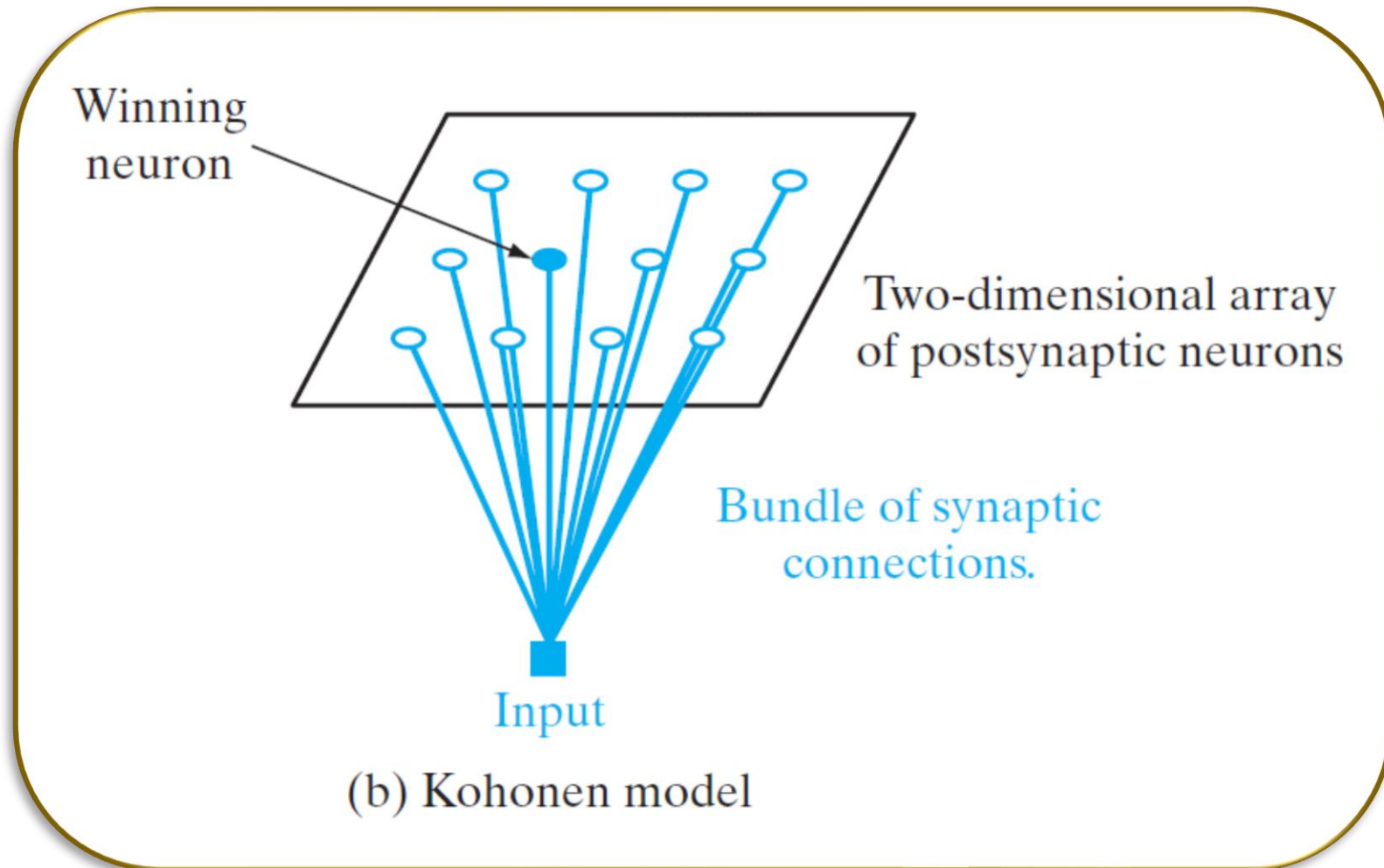
- Different sensory inputs (motor, somatosensory, visual, auditory, etc.) are mapped onto corresponding areas of the cerebral cortex in an **orderly fashion**.
- Computational maps offer four properties:
 - P1: In each map, neurons act in parallel and process pieces of information that are similar in nature, but originate from different regions in the sensory input space.
 - P2: At each stage of representation, each incoming piece of information is kept in its proper context.
 - P3. Neurons dealing with closely related pieces of information are close together so that they can interact via short synaptic connections.
 - P4. Contextual maps can be understood in terms of decision-reducing mappings from higher-dimensional parameter spaces onto the cortical surface.

- As a neural network developer, what are you interest?
 - **Building artificial topographic maps that learn through self-organization in a neurobiologically inspired manner.**
- Computational mapping of the brain results in following principle of topographic map formation, stated as follows (Kohonen, 1990):
 - **The spatial location of an output neuron in a topographic map corresponds to a particular domain or feature of data drawn from the input space.**
- *This principle has provided the neurobiological motivation for **two different feature-mapping models.***

Feature Mapping Model - 1



Feature Mapping Model - 2

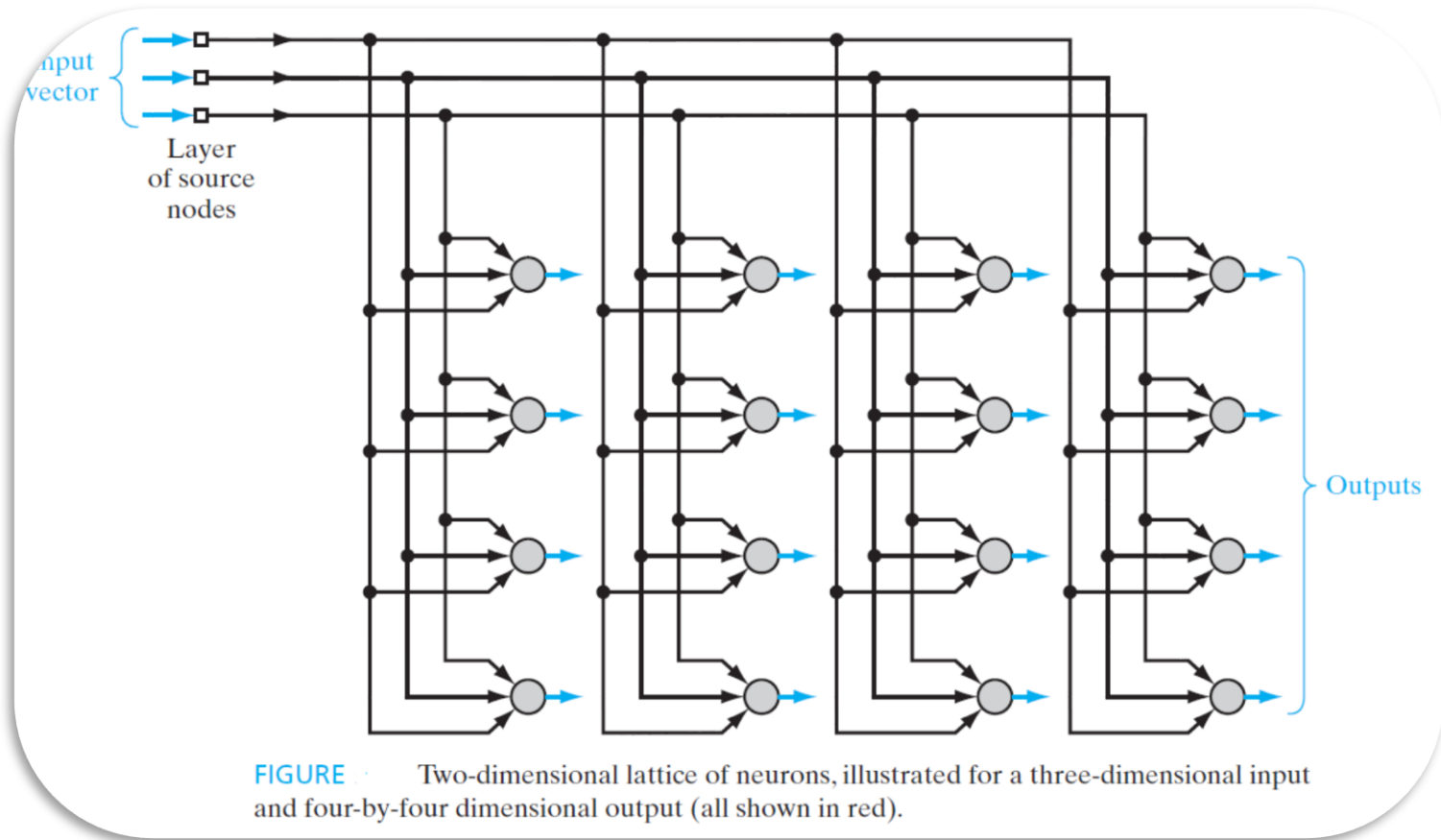


Kohonen's Model

- The model captures the essential features of computational maps in the brain and yet remains computationally tractable. It appears that the Kohonen model is more general than the Willshaw–von der Malsburg model in the sense that it is capable of performing data compression (i.e., dimensionality reduction on the input).
- The Kohonen model belongs to the class of vector-coding algorithms.
- The model provides a topological mapping that optimally places a fixed number of vectors (i.e., code words) into a higher-dimensional input space, thereby facilitating data compression.

Kohonen's Model

- The principal goal of the self-organizing map (SOM) is to transform an incoming signal pattern of arbitrary dimension into a one- or two-dimensional discrete map, and to perform this transformation adaptively in a topologically ordered fashion.



Algorithm Self-Organize;

- Select network topology to determine which nodes are adjacent to which others;
- Initialize weights to small random values;
- Initialize current neighborhood distance $D(0)$ to a positive integer;
- **while** computational bounds are not exceeded, **do**
 1. Select an input sample i_ℓ ;
 2. Compute the square of the Euclidean distance of i_ℓ from the weight vector (w_j) associated with each output node:
$$\sum_{k=1}^n (i_{\ell,k}(t) - w_{j,k}(t))^2;$$
 3. Select the output node j^* with minimum $\sum_{k=1}^n (i_{\ell,k}(t) - w_{j,k}(t))^2$;
 4. Update weights to all nodes within a topological distance of $D(t)$ from j^* , using the update rule
$$w_j(t+1) = w_j(t) + \eta(t)(i_\ell(t) - w_j(t)),$$
where $0 < \eta(t) \leq \eta(t-1) \leq 1$;
 5. Increment t ;**end-while.**

- Cluster the following points using **Self Organizing Maps**.
- Assume $c = 0.5$.
- Input vectors $X1 = [0 \ 1 \ 1]$ $X2 = [1 \ 1 \ 0.5]$
 $X3 = [0.6 \ 0.3 \ 0.4]$ $X4 = [0.5 \ 0.6 \ 0.7]$
 $X5 = [0.7 \ 0.4 \ 0.4]$
- $W = [0.14 \ 0.75 \ 0.71; \quad 0.99 \ 0.51 \ 0.37; \quad 0.73 \ 0.81 \ 0.87]$

Step 1: Perform Initialization

- Learning constant $c = 0.5$.

- Input vectors $X1 = [0 \ 1 \ 1]$ $X2 = [1 \ 1 \ 0.5]$
 $X3 = [0.6 \ 0.3 \ 0.4]$ $X4 = [0.5 \ 0.6 \ 0.7]$
 $X5 = [0.7 \ 0.4 \ 0.4]$

- Initial Weight vectors

$W = [0.14 \ * \ 0.75 \ * \ 0.71; \quad 0.99 \ * \ 0.51 \ * \ 0.37; \quad 0.73 \ * \ 0.81 \ * \ 0.87]$

Initialize the required parameters

**Compute Euclidean Distances
between each Input Vector X1 and
the weight vectors W1 W2 W3**

Select the Winner

**Update the weight of WINNING AS
WELL AS NEIGHBOURING
Vectors**

**Repeat these 4 steps for
X2 X3 X4 X5 and
update the weights
respectively.**

**If required change 'c'.
Iteration 1 is complete**

Step 2: Compute the distance between each input vector and each weight vector using Euclidean distance.

- For input vector $X1 = [0 \ 1 \ 1]$
- $D(X1, W1) = \sqrt{(0 - 0.14)^2 + (1 - 0.75)^2 + (1 - 0.71)^2} = \sqrt{0.1662} = 0.4076$
- $D(X1, W2) = 1.272$
- $D(X1, W3) = 0.765$

Step 3: Best Matching Unit (BMU)

- For $X1 = [0 \ 1 \ 1]$
- The closest weight vector is $W1$.
- Therefore, $W1$ is the BMU for $X \ 1$

Step 4: Update Weight for winning neuron as well as its neighbours.

$$w(t+1) = w(t) + \eta(t) \times (x - w(t))$$

- $w1_new = w1_old + c * (X1 - w1_old)$
- $w1_new = [0.14 \ 0.75 \ 0.71] + (0.5 * ([0 \ 1 \ 1] - [0.14 \ 0.75 \ 0.71]))$
- $w1_new = [0.07 \ 0.875 \ 0.855]$
- $w2_new = w2_old + c * (X1 - w2_old)$
- $w2_new = [0.495 \ 0.755 \ 0.685]$
- $w3_new = w3_old + c * (X1 - w3_old)$
- $w3_new = [0.365 \ 0.905 \ 0.935]$

Considering X2

- For input vector $X2 = [1 \ 1 \ 0.5]$
- Distance between
- $D(X2, W1) = 1.0032$
- Winner
- $D(X2, W2) = 0.4195$
- $D(X2, W3) = 0.78$
- Updated weight Matrix
- $W1_new = [0.535, \ 0.9375 \ 0.6775]$
- $W2_new = [0.7475 \ 0.8775 \ 0.5925]$
- $W3_new = [0.6825 \ 0.9525 \ 0.7175]$

Considering X3

- Input $X3 = [0.6 \ 0.3 \ 0.4]$
 - Winning vector = W2
 - Updated Weights:
 - $W1_new = [0.5675 \ 0.61875 \ 0.53875]$
 - $W2_new = [0.67375 \ 0.58875 \ 0.49625]$
 - $W3_new = [0.64125 \ 0.62625 \ 0.55875]$
- $D(X3, W1) = 0.6987$
 - $D(X3, W2) = 0.6265$
 - $D(X3, W3) = 0.7308$

Considering X4

- Input $X4 = [0.5 \ 0.6 \ 0.7]$

- Winning vector = W1

- Updated Weights:

- $W1_new = [0.53375 \ 0.609375 \ 0.619375]$

- $W2_new = [0.586875 \ 0.594375 \ 0.598125]$

- $W3_new = [0.570625 \ 0.613125 \ 0.629375]$

- $D(X4, W1) = 0.1759$

- $D(X4, W2) = 0.268$

- $D(X4, W3) = 0.2014$

Considering X5

- Input $X5 = [0.7 \ 0.4 \ 0.4]$
 - Winning vector = W2
 - Updated Weights:
 - $W1_new = [0.616875 \ 0.5046875 \ 0.5096875]$
 - $W2_new = [0.6434375 \ 0.4971875 \ 0.4990625]$
 - $W3_new = [0.6353125 \ 0.5065625 \ 0.5146875]$
- $D(X5, W1) = 0.3459$
 - $D(X5, W2) = 0.2999$
 - $D(X5, W3) = 0.3389$

At the end of 1st Iteration

- Final Weight vectors
 - $W1_new = [0.616875 \quad 0.5046875 \quad 0.5096875]$
 - $W2_new = [0.6434375 \quad 0.4971875 \quad 0.4990625]$
 - $W3_new = [0.6353125 \quad 0.5065625 \quad 0.5146875]$
- Cluster 1 = X1 X4
- Cluster 2 = X2 X3 X5
- Cluster 3 = -----

Advantages of SOM

- Topology Preservation:**

- SOMs preserve the neighborhood relationships of the input data. This means that similar inputs are mapped close to each other on the output grid, which is useful for clustering and visualization.

- Dimensionality Reduction:**

- SOMs reduce high-dimensional data into a low-dimensional space (often 2D), making it easier to visualize complex data.

- Unsupervised Learning:**

- SOMs do not require labeled data, which is useful when dealing with large datasets where labeling might be expensive or infeasible.

Advantages of SOM

- Data Clustering:**

- SOMs can cluster data efficiently based on similarity without needing explicit category labels. This helps in segmenting datasets into meaningful groups.

- Adaptation to Nonlinear Data:**

- They are capable of handling complex, nonlinear relationships between variables in the input data.

Disadvantages of SOM

- **Training Complexity:**
 - The training process of SOMs can be computationally intensive, especially for large datasets, as the algorithm needs to iteratively adjust the weights of the network.
- **Lack of Theoretical Foundation:**
 - While SOMs work well in practice, there is a lack of a strong theoretical basis compared to other methods like Principal Component Analysis (PCA), making it harder to predict performance or set parameters optimally.
- **Curse of Dimensionality:**
 - SOMs do not always perform well with very high-dimensional data. As the dimensionality increases, the distances between data points tend to become more uniform, making it difficult to maintain neighborhood relationships.

Disadvantages of SOM

- **Interpretability:**
- Although SOMs are useful for clustering, interpreting the meaning of clusters and mapping back from the 2D grid to the original data space can be challenging.
- **No Objective Function:**
- Unlike other clustering algorithms (e.g., k-means), SOMs lack a well-defined objective function to minimize, making convergence and optimization more challenging.
- **Sensitive to Hyper-parameters:**
- SOMs require the careful tuning of hyperparameters (e.g., learning rate, neighborhood size). Improper tuning can lead to poor performance or failure to capture important data structures.

Adaptive Resonance Theory

Steps of the ART algorithm

- **Initialization:** Initialize weights and choose a vigilance parameter ρ .
- **Similarity Measure:** Compare input vectors with weight vectors using a similarity or matching criterion.
- **Resonance Test:** Check if the match meets the vigilance criterion.
- **Weight Update:** If a resonant match is found, update the weight vector. If no match is found, create a new weight vector.

Adaptive Resonance Theory

Algorithm ART1:

Initialize each top-down weight $t_{\ell,j}(0) = 1$;
Initialize each bottom-up weight $b_{j,\ell}(0) = \frac{1}{n+1}$;
while the network has not stabilized, do

1. Present a randomly chosen pattern $x = (x_1, \dots, x_n)$ for learning;
2. Let the active set A contain all nodes; calculate $y_j = b_{j,1} x_1 + \dots + b_{j,n} x_n$ for each node $j \in A$;
3. repeat
 - (a) Let j^* be a node in A with largest y_j , with ties being broken arbitrarily;
 - (b) Compute $s_A = (s_1^*, \dots, s_n^*)$ where $s_\ell^* = t_{\ell,j^*} x_\ell$;
 - (c) Compare similarity between s^* and x with the given vigilance parameter ρ :
If $\frac{\sum_{\ell=1}^n s_\ell^*}{\sum_{\ell=1}^n x_\ell} \leq \rho$ then remove j^* from set A
else associate x with node j^* and update weights:

$$b_{j^*,\ell}(\text{new}) = \frac{t_{\ell,j^*}(\text{old}) x_\ell}{0.5 + \sum_{\ell=1}^n t_{\ell,j^*}(\text{old}) x_\ell}$$

$$t_{\ell,j^*}(\text{new}) = t_{\ell,j^*}(\text{old}) x_\ell$$

until A is empty or x has been associated with some node j ;

4. If A is empty, then create a new node whose weight vector coincides with the current input pattern x ;

end-while.

- **Top-Down Weights** represent the expectations of the network. They reflect the network's template for the input pattern associated with a specific node or category. The top-down weight vector allows the network to check whether an input pattern matches the learned prototype for a category.
- During learning, top-down weights are updated to become more similar to the input pattern whenever a match occurs between the input and a category.
- Initially, these weights are broad and cover a wide range of patterns.
- As learning progresses, the top-down weights are fine-tuned based on the inputs that have been associated with each category.

- When a new input pattern is presented, the network uses top-down weights to perform a template matching process.
- It compares the current input with the top-down weight vector for each node to see how similar the input is to a known category.
- This is done using a vigilance test, where the similarity (matching strength) is compared to a vigilance parameter (ρ).
- If the similarity is too low, the pattern does not belong to the category, and the search continues. Otherwise, the category is considered a match, and learning can occur.

- **Bottom-up weights** - control the network's activation in response to an input pattern. They act as feedforward signals from the input layer to the nodes (categories) and determine how strongly a node is activated by a given input.
- Each bottom-up weight vector represents how strongly each feature (component) of the input pattern contributes to the activation of the corresponding node.
- These weights are updated after a node is chosen, helping the network refine its sensitivity to specific input patterns.
- Bottom-up weights help determine the degree of overlap between an input pattern and the weight vector of a node. The higher the overlap, the higher the activation y_j for that node.

- The bottom-up weights are used to calculate the node activations y_j .
- The node with the highest activation value is selected for the next step in the algorithm, where it is compared to the input for possible learning.
- This mechanism ensures that the most relevant node (the one whose bottom-up weights most closely align with the input) is chosen for further comparison.

Top-down weights $t_{i,j}(0) = 1$: These are initialized to 1 for each node j and input feature i .

Bottom-up weights $b_{j,\ell}(0) = \frac{1}{n+1}$: These weights are initialized to a small value that depends on the number of features n , where ℓ indexes the input feature.

Bidirectional Interaction

- In ART learning happen through a bidirectional process:
- Bottom-up weights: Initiate the recognition process by selecting the most likely candidate category (node) for an input.
- Top-down weights: Refine this process by checking if the selected category sufficiently matches the input pattern, based on the vigilance parameter.
- ART networks must balance plasticity (the ability to learn new patterns) with stability (the ability to retain old patterns). The dual-weight structure helps achieve this balance.
 - Top-down weights are responsible for ensuring stability: they prevent the network from "forgetting" previously learned categories by gradually fine-tuning prototypes (templates).
 - Bottom-up weights are responsible for learning and plasticity: they allow the network to adapt quickly to new inputs and ensure that categories are selected based on input similarity.

Pattern Presentation

A randomly chosen input pattern $x = (x_1, \dots, x_n)$

Active Set Calculation

$$y_j = b_{j,1}x_1 + \dots + b_{j,n}x_n$$

Repeat until Learning completes

(a) **Choosing the Best Matching Node:** - Let j^* be the node in the active set A that has the highest y_j . If there is a tie, it is broken arbitrarily.

(b) **Calculate Similarity:** - Compute the similarity vector s_A for node j^* :
 $s_A = (s_1, \dots, s_n)$ where $s_i = t_{i,j^*} \cdot x_i$ This calculates how similar the input x is to the stored pattern in node j^* .

(c) **Comparison with Vigilance Parameter ρ :** - Calculate the similarity between s_A and the input pattern x . If the similarity, computed as: $\frac{\sum_{i=1}^n s_i}{\sum_{i=1}^n x_i} \leq \rho$ is less than or equal to the vigilance parameter ρ , then the node j^* is removed from the active set A , and the algorithm repeats with the next highest y_j .

Weight Update

Bottom-up weights $b_{j^*,\ell}$ are updated as:

$$b_{j^*,\ell}(\text{new}) = \frac{t_{j^*,\ell}(\text{old}) \cdot x_\ell}{0.5 + \sum_{t=1}^n t_{j^*,t}(\text{old}) \cdot x_t}$$

This formula updates the bottom-up weight using a normalization scheme based on the overlap between the input and the previous pattern.

Top-down weights t_{i,j^*} are updated as:

$$t_{i,j^*}(\text{new}) = t_{i,j^*}(\text{old}) \cdot x_i$$

This updates the top-down weight to store the current input pattern more closely.

Creation of a New Node:

If the active set A becomes empty and the input x is not yet associated with any node, a new node is created, and its weight vector is set to the current input pattern.

a) Apply ART for one full iteration and determine the final weights. Assume $\rho = 0.8$

$X_1 = (1, 0, 1, 0, 0, 0, 1)$, $X_2 = (0, 0, 1, 1, 1, 1, 0)$, $X_3 = (1, 1, 0, 0, 0, 0, 1)$

$X_4 = (0, 0, 0, 1, 1, 1, 1)$, $X_5 = (1, 1, 1, 0, 0, 0, 0)$.

- $X_1 = (1, 0, 1, 0, 0, 0, 1)$ $X_2 = (0, 0, 1, 1, 1, 1, 0)$
- $X_3 = (1, 1, 0, 0, 0, 0, 1)$ $X_4 = (0, 0, 0, 1, 1, 1, 1)$
- $X_5 = (1, 1, 1, 0, 0, 0, 0)$
- Vigilance parameter $\rho = 0.8$

- **Top-down weights** $t_{i,j}(0) = 1$ for all nodes j and all input features i .
- **Bottom-up weights** $b_{j,k}(0) = 1 / (n+1) = 1 / (7+1) = 0.125$
- **Present** $X1 = (1, 0, 1, 0, 0, 0, 1)$ for learning.
- Create **Node 1** and randomly assign $X1$ as its pattern.
- Calculate y_j for each **Node 1** using the **initial bottom-up weights**:
- $y_j = (b_{j,1} * x_1) + (b_{j,2} * x_2) + (b_{j,3} * x_3) + (b_{j,4} * x_4) + (b_{j,5} * x_5) + (b_{j,6} * x_6) + (b_{j,7} * x_7)$
- $y_j = 0.125 * (1+0+1+0+0+0+1) = 0.125 * 3$
- $y_j = 0.375$

- **Compute the similarity between X1 and the stored pattern in Node 1.**

- $S_A = (1 * 1, 1 * 0, 1 * 1, 1 * 0, 1 * 0, 1 * 0, 1 * 1)$

- $S_A = (1, 0, 1, 0, 0, 0, 1)$

- Calculate the similarity measure: $\frac{\sum_{i=1}^n s_i}{\sum_{i=1}^n x_i}$

- Similarity measure = $3 / 3 = 1$

- Since $1 \geq \rho$, i.e. $1 \geq 0.8$,

→ we associate the input pattern X1 with Node 1.

Update weights for Node 1

- Update bottom-up weights

$$b_{1,\ell}(\text{new}) = \frac{t_{1,\ell}(\text{old}) \cdot x_{\ell}}{0.5 + \sum_{t=1}^n t_{1,t}(\text{old}) \cdot x_t}$$

- $b_{1,L} = (1 * X_L) / (0.5 + [1 * 3])$

- $b_{1,L} = (X_L / 3.5)$

- New bottom-up weights are:

- $(1/3.5 , 0/3.5 , 1/3.5 , 0/3.5 , 0/3.5 , 0/3.5 , 1/3.5)$ or

- $(0.2857 , 0 , 0.2857 , 0 , 0 , 0 , 0.2857)$

Update weights for Node 1

- Update top-down weights:

$$t_{\ell,j}(\text{new}) = t_{\ell,j}(\text{old})x_{\ell}$$

- $t_{i,j1}(\text{new}) = (1, 0, 1, 0, 0, 0, 1)$

- Comment: "X1 is in Node 1"

For Node X2 (0 , 0 , 1 , 1 , 1 , 1 , 0)

- Calculate y_j for each Node 1 using the **updated bottom-up weights**:
- $y_j = (b_{j,1} * x_1) + (b_{j,2} * x_2) + (b_{j,3} * x_3) + (b_{j,4} * x_4) + (b_{j,5} * x_5) + (b_{j,6} * x_6) + (b_{j,7} * x_7)$
- Updated Weight Vector: (0.2857 , 0 , 0.2857 , 0 , 0 , 0 , 0.2857)
- $y_j = ([0 * 0.286] + [0 * 0] + [1 * 0.286] + [1 * 0] + [1 * 0] + [1 * 0] + [0 * 0.268])$
- $y_j = 0.286$

- **Compute the similarity between X2 and the stored pattern in Node 1.**

- $S_A = (1 * 0, 0 * 0, 1 * 1, 0 * 1, 0 * 1, 0 * 1, 1 * 0)$

- $S_A = (0, 0, 1, 0, 0, 0, 0)$

- Calculate the similarity measure: $\frac{\sum_{i=1}^n s_i}{\sum_{i=1}^n x_i}$

- Similarity measure = $1 / 4 = 0.25$

- Since $0.25 \not\geq 0.8$,

→ we DO NOT associate the input pattern X2 with Node 1.

Update weights for Node 1

- Update bottom-up weights

$$b_{1,\ell}(\text{new}) = \frac{t_{1,\ell}(\text{old}) \cdot x_\ell}{0.5 + \sum_{t=1}^n t_{1,t}(\text{old}) \cdot x_t}$$

- $b_{1,L} = (1 * X_L) / (0.5 + [1 * 3])$

- $b_{1,L} = (X_L / 1.5)$

- New bottom-up weights are:

- $(1/3.5 , 0/3.5 , 1/3.5 , 0/3.5 , 0/3.5 , 0/3.5 , 1/3.5)$ or

- $(0.2857 , 0 , 0.2857 , 0 , 0 , 0 , 0.2857)$

Update weights for Node 1

- Update top-down weights:

$$t_{\ell,j}(\text{new}) = t_{\ell,j}(\text{old})x_{\ell}$$

- $t_{i,j1}(\text{new}) = (0, 0, 1, 1, 1, 1, 0)$

- Comment: "X2 is in Node 1"

Advantages of ART

- **Stability-Plasticity Balance:**
- ART networks excel at balancing stability (retaining previously learned patterns) and plasticity (learning new patterns). This ensures that the network can learn new data without forgetting old knowledge, a major challenge in other neural networks.
- **Incremental Learning:**
- ART networks can perform online learning, meaning they can continuously learn and adapt to new data without needing to retrain from scratch. This makes them highly useful in dynamic environments where data arrives over time.

Advantages of ART

- **Unsupervised and Supervised Learning:**
 - ART models can be used in both unsupervised (e.g., clustering) and supervised learning tasks (e.g., classification), making them versatile in various applications.
- **Resistance to Catastrophic Forgetting:**
 - Unlike traditional neural networks, which can suffer from catastrophic forgetting, ART networks retain old patterns while learning new ones.

Advantages of ART

- **Self-regulation:**

- ART networks automatically regulate their complexity, meaning they can adjust the number of categories or clusters dynamically as new input data is presented. This is especially beneficial in environments where the number of classes or clusters is not known in advance.

- **Robustness to Noise:**

- ART networks are relatively robust to noisy data because they have mechanisms for filtering out noise and focusing on the core patterns in the data.

- **Fast Learning:**

- Learning in ART networks is typically fast because they only update representations when necessary (based on resonance criteria). If the new data resonates with an existing category, no change is made, leading to faster convergence.

Disadvantages of ART

- **Complexity of Network Design:**
- ART networks have a complex structure and require careful parameter tuning, particularly the vigilance parameter, which controls the granularity of learning. Setting this parameter incorrectly can lead to poor performance, either underfitting or overfitting.
- **Sensitivity to Vigilance Parameter:**
- The vigilance parameter, which determines how similar a new input must be to an existing category for resonance to occur, is highly sensitive. A low vigilance value may cause the network to group dissimilar patterns together, while a high value may lead to too many categories being formed.

Disadvantages of ART

- **High Resource Consumption:**
- ART networks can be computationally expensive, particularly in high-dimensional data spaces, due to the need for comparing each new input with all existing categories in the network.
- **Over-Clustering or Under-Clustering:**
- Due to the sensitivity of the vigilance parameter, ART networks might either over-cluster making it challenging to find the optimal solution.

Disadvantages of ART

- While ART was inspired by biological processes, some aspects of its behavior, such as the sharpness of categorical decisions based on the vigilance parameter, may not align with how biological systems process and adapt to information.
- **Difficult Interpretation:**
- Similar to other neural networks, interpreting the learned categories and understanding the internal structure of the ART network can be difficult, especially when the number of categories becomes large or complex.